

## TASK 3 – Derive the Time-Complexity Function Mathematically

Let  $n$  = number of student records.

```
□ genui □ {"learning_viz":{"type_id":"BIG_O_TIME_COMPLEXITY","local_override":"en-IN"}} □
```

### 1. Linear Search

#### Input size

Let there be  **$n$  student records**:

records[0], records[1], ..., records[n-1]

#### Basic operation

The basic operation is:

records[i] == key

#### Number of operations

In the **worst case**, the required registration number is either the last record or is not present.

Therefore, the algorithm performs:

$1 + 1 + 1 + \dots + 1$

$n$  times

So,

$$C(n) = n$$

If each comparison takes constant time  $c$ , then:

$$T(n) = cn + c_0$$

where  $c_0$  represents other constant operations.

Therefore:

$$T(n) = \Theta(n)$$

### **Preprocessing cost**

No sorting or restructuring is required.

$$P(n) = 0$$

### **Overall cost**

$$T_{total}(n) = P(n) + T_{search}(n)$$

$$T_{total}(n) = 0 + \Theta(n)$$

$$\boxed{T_{total}(n) = \Theta(n)}$$

## **2. Binary Search — Recursive**

### **Input size**

There are **n sorted student records**.

### **Preprocessing**

Binary search requires the records to be sorted.

If the records are initially unsorted, sorting requires:

$$P(n) = \Theta(n \log n)$$

### **Basic operation**

The main operation is comparing the target with the middle element.

At every recursive call, approximately half of the records are eliminated.

Therefore:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \dots$$

After k steps:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

So the number of comparisons is approximately:

$$C(n) = \log_2 n$$

The running time can therefore be written as:

$$T(n) = c \log_2 n + c_0$$

Hence:

$$\boxed{T(n) = \Theta(\log n)}$$

### **Overall cost including sorting**

If sorting is required:

$$T_{total}(n) = \Theta(n \log n) + \Theta(\log n)$$

Since  $n \log n$  grows much faster than  $\log n$ :

$$\boxed{T_{total}(n) = \Theta(n \log n)}$$

**Important:** If the university keeps the records sorted continuously, the sorting cost does not need to be paid for every search.

## **3. Hash Search**

### **Input size**

Let there be **n student records** stored in a hash table.

### **Preprocessing**

The records must first be inserted into the hash table.

For  $n$  records, the insertion process requires approximately  $n$  operations on average:

$$P(n) = \Theta(n)$$

### Basic operation

The main operation during searching is:

$$hashTable[index] == key$$

First, the hash function calculates the location:

$$index = key \bmod m$$

where  $m$  is the hash-table size.

If there is no collision, only approximately **one comparison** is needed.

Therefore:

$$C(n) \approx 1$$

So the average search time is:

$$T_{search}(n) = \Theta(1)$$

### Overall cost

Including construction of the hash table:

$$\begin{aligned} T_{total}(n) &= P(n) + T_{search}(n) \\ T_{total}(n) &= \Theta(n) + \Theta(1) \end{aligned}$$

Therefore:

$$\boxed{T_{total}(n) = \Theta(n)}$$

However, **after the hash table has already been constructed**, each search is approximately:

$$\boxed{\Theta(1)}$$

on average.

| Algorithm     | Preprocessing                  | Search Operations   | Search Complexity   | Total Including Preprocessing |
|---------------|--------------------------------|---------------------|---------------------|-------------------------------|
| Linear Search | 0                              | $n$                 | $\Theta(n)$         | $\Theta(n)$                   |
| Binary Search | $\Theta(n \log n)$ sorting     | $\log_2 n$          | $\Theta(\log n)$    | $\Theta(n \log n)$ initially  |
| Hash Search   | $\Theta(n)$ table construction | $\approx 1$ average | $\Theta(1)$ average | $\Theta(n)$ initially         |

## Conclusion for Task 3

As the number of student records increases, **linear search grows directly with  $n$** , while **binary search grows logarithmically after sorting**. Hash search provides approximately **constant-time searching after the hash table has been constructed**. Therefore, preprocessing must be considered when comparing the algorithms, especially for the initial construction of the data structure.

